

Множење матрица - Control Flow Manuscopre парадигма, Si имплементациона технологија

Електротехнички факултет, Универзитет у Београду

Никола Николић 2025/3109

Павле Шаренац 2025/3068

Control Flow Manycore парадигма

- *Control Flow Multicore* – 4-32 моћна језгра опште намене
- *Control Flow Manycore* – хиљаде слабијих језгара
- Циљ није општа намена, већ масовна паралелизација обраде велике количине података

NVIDIA GPU

- NVIDIA GPU (*Graphical Processing Unit*) је одличан пример хардвера који користи manycore парадигму
- Хиљаде CUDA језгара груписаних у SM (*Streaming Multiprocessors*)
- SIMT (*Single Instruction Multiple Threads*) извршавање
- SM-ови извршавају истовремено велики број *warp*-ова нити, односно групе од по 32 нити

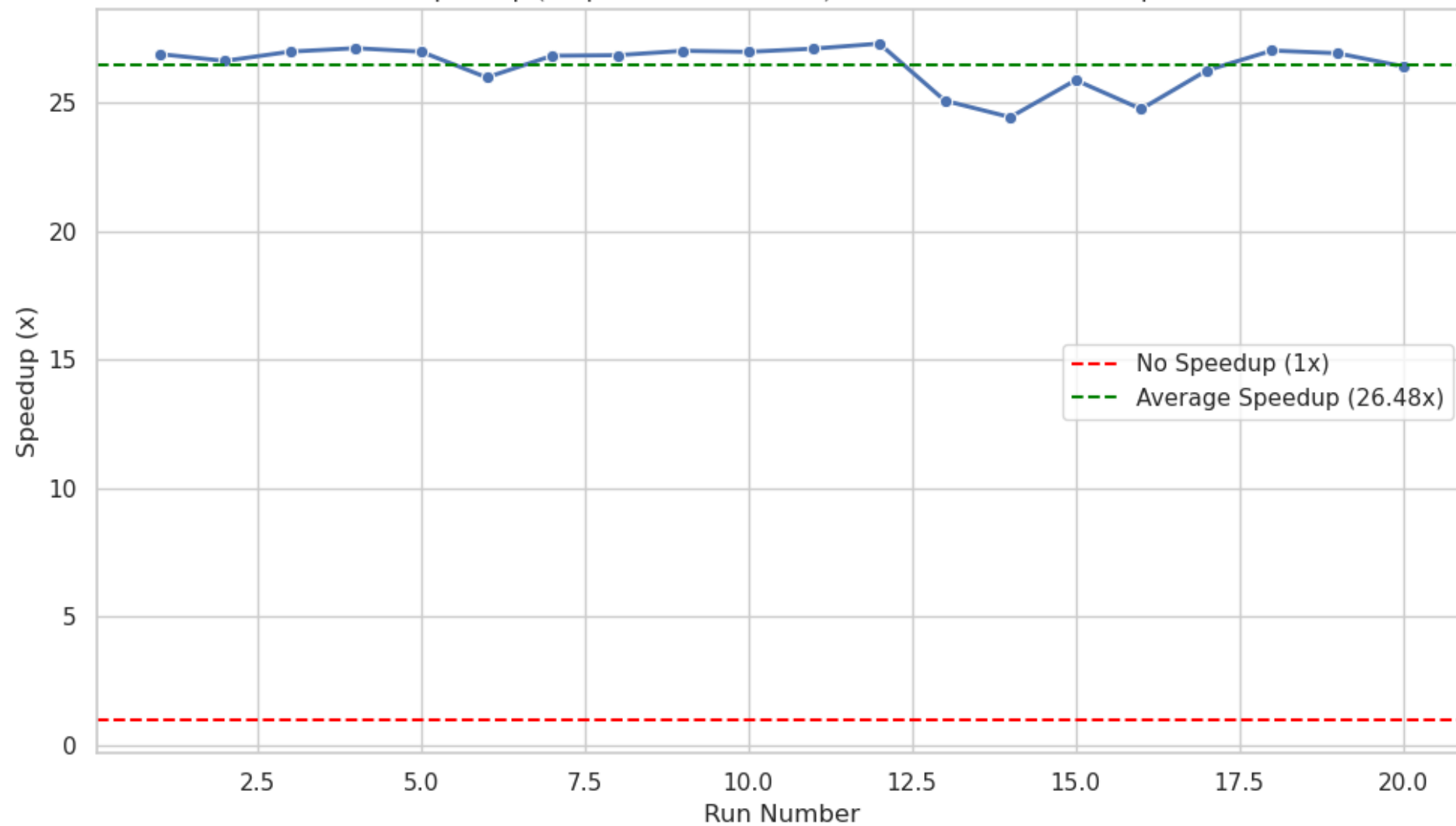
CUDA програмски модел

- Постоје *Host* (CPU) и *Device* (GPU) одвојени меморијски простори
- Трансфер података са CPU на GPU је неопходан - спорија операција, приметна на мањим проблемима, не на већим
- Хијерархија нити на GPU: *Grid* → *Block* → *Thread*
- Меморијска хијерархија: глобална меморија (споро читање, ~400 циклуса) → дељена меморија (брзо читање, ~5 циклуса, приватна по блоку нити) → регистри (најбрже читање, приватни за сваку нит)
- Кернел функција (**__global__**) дефинише задатак једне нити, а GPU извршава ову функцију на хиљадама нити у паралели

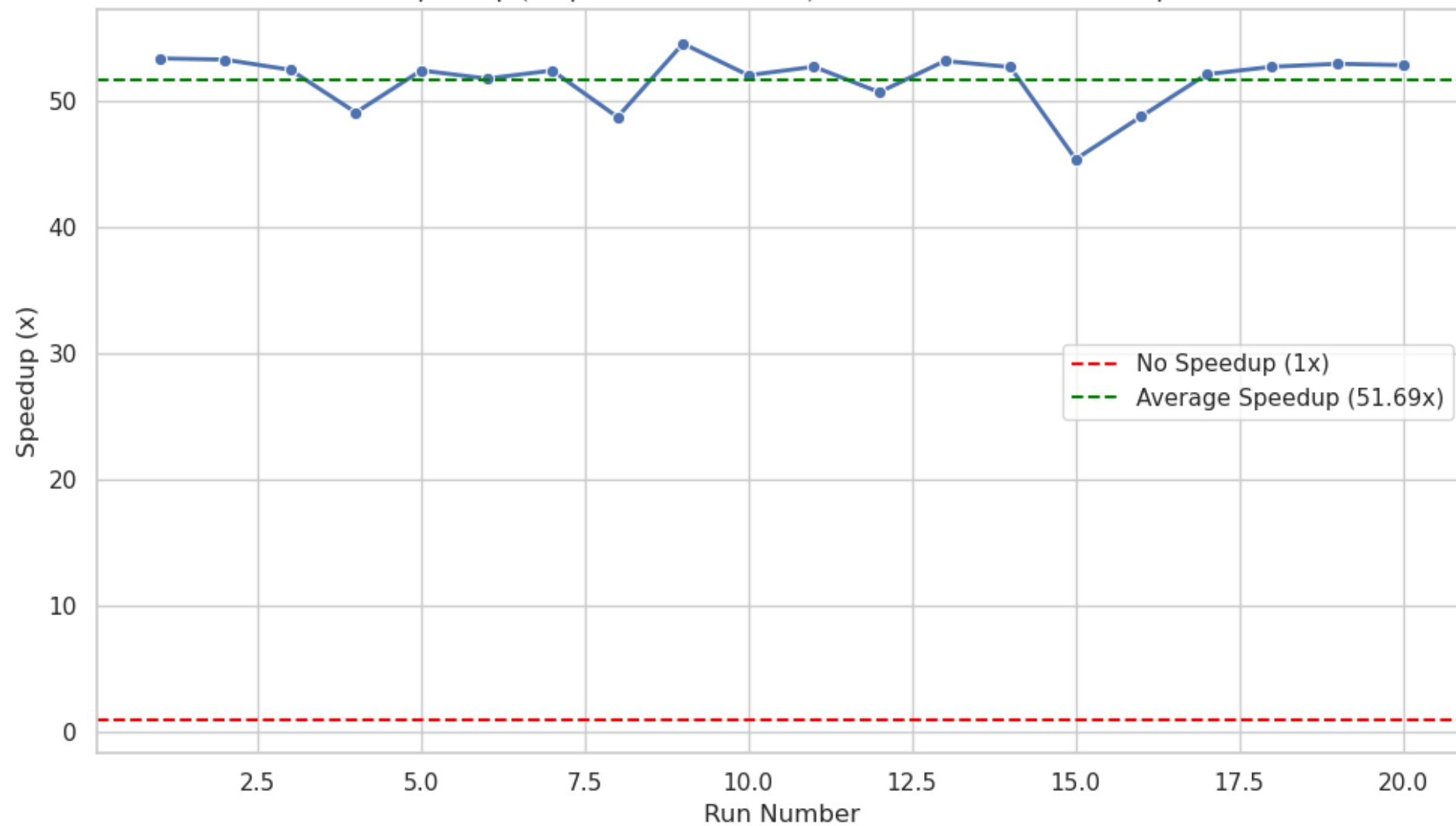
Множење матрица - CUDA кернел

```
global void matmul_tiled_kernel(const double * __restrict__ A, const double * __restrict__ B, double * __restrict__ C, int N)
{
    __shared__ double tile_A[TILE_SIZE][TILE_SIZE];
    __shared__ double tile_B[TILE_SIZE][TILE_SIZE];
    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    double partial_sum = 0.0;
    int number_of_tiles = (N + TILE_SIZE - 1) / TILE_SIZE;
    for (int tile_index = 0; tile_index < number_of_tiles; tile_index++)
    {
        int a_col = tile_index * TILE_SIZE + threadIdx.x;
        int b_row = tile_index * TILE_SIZE + threadIdx.y;
        tile_A[threadIdx.y][threadIdx.x] = (row < N && a_col < N)
            ? A[row * N + a_col]
            : 0.0;
        tile_B[threadIdx.y][threadIdx.x] = (b_row < N && col < N)
            ? B[b_row * N + col]
            : 0.0;
        __syncthreads();
        for (int k = 0; k < TILE_SIZE; k++)
        {
            partial_sum += tile_A[threadIdx.y][k] * tile_B[k][threadIdx.x];
        }
        __syncthreads();
    }
    if (row < N && col < N)
    {
        C[row * N + col] = partial_sum;
    }
}
```

Per-Run Speedup (Sequential vs. Parallel) — 512x512 Matrix Multiplication



Per-Run Speedup (Sequential vs. Parallel) — 1024x1024 Matrix Multiplication



Per-Run Speedup (Sequential vs. Parallel) — 2048x2048 Matrix Multiplication

